OR114-2 Midterm Project

Problem 1

1. Mathematical Formulation

Sets

- C : Set of vehicles.
- O : Set of customer orders.
- p_i : Pickup station of order i.
- r_i : Return station of order i.
- s_i : Pick-up time of order i.
- e_i : Return time of order i.
- l_i : Requested vehicle level for order i.
- R_i : Revenue generated by order i.
- h_c : Initial station of vehicle c.
- q_c : Vehicle level of vehicle c.

Parameters

- A^0: set of feasible initial assignments
- A: set of feasible order transitions
- m_{ci}: relocation time from vehicle c to order i
- m_{ij}: relocation time between orders i and j
- B: relocation time budget

The feasible sets A^0 and A are defined by IEDO timing constraints, where a transition (i, j) is valid only if:

$$RE_i + 240 + T_{ab} \leq PU_j - 30$$

where RE_i, T_{ab}, and PU_j represent return time, travel time, and pickup time, respectively.

Decision Variables

$$y_i = \begin{cases} 1, & \text{if order } i \text{ is accepted} \\ 0, & \text{otherwise} \end{cases}$$

$$x_{ci} = \begin{cases} 1, & \text{if vehicle } c \text{ starts with order } i \\ 0, & \text{otherwise} \end{cases}$$

$$z_{cij} = \begin{cases} 1, & \text{if vehicle } c \text{ serves order } j \text{ after } i \\ 0, & \text{otherwise} \end{cases}$$

Objective Function

$$\max \sum_{i \in O} R_i y_i$$

Constraints

(1) Order assignment constraint

$$\sum_{(c,i)\in A^0} x_{ci} + \sum_{(c,j,i)\in A} z_{cji} = y_i \qquad \forall i \in O$$

(2) Vehicle starting constraint

$$\sum_{i:(c,i)\in A^0} x_{ci} \leq 1 \qquad \forall c \in C$$

(3) Flow conservation constraint

$$\sum_{j:(c,i,j)\in A} z_{cij} \leq x_{ci} + \sum_{k:(c,k,i)\in A} z_{cki} \qquad \forall c \in C, i \in O$$

(4) Relocation budget constraint

$$\sum_{(c,i)\in A^0} m_{ci} x_{ci} + \sum_{(c,i,j)\in A} m_{ij} z_{cij} \leq B$$

2. Optimal Solution for Instance 05

The optimization model accepts 8 out of 10 orders, including Orders 3, 4, 5, 6, 7, 8, 9, and 10. Orders 1 and 2 are rejected. The accepted sales revenue is 117,000, and the final profit after rejection compensation costs is 106,800. The total relocation time used is 1,080 minutes, which satisfies the relocation budget limit ($1080 \leq 1200$). Only one vehicle upgrade is required in the final solution, where a level-2 vehicle is assigned to Order 6, which requests a level-1 vehicle.

3. Operational Plan

Order	Car	Upgrade	Pickup → Return	Rental Period	Revenue
3	3	No	5 → 1	01/02 03:30 – 01/04 11:30	5,600
4	1	No	1 → 2	01/03 12:00 – 01/05 15:00	5,100
5	6	No	6 → 6	01/01 00:00 – 01/04 20:00	36,800
6	4	Yes	3 → 4	01/04 23:00 – 01/05 14:00	1,500
7	5	No	2 → 1	01/02 03:30 – 01/04 23:30	27,200
8	6	No	6 → 6	01/05 04:30 – 01/05 14:30	4,000
9	4	No	3 → 4	01/01 19:30 – 01/04 15:30	27,200
10	2	No	4 → 2	01/01 06:00 – 01/05 06:00	9,600

4. Vehicle Relocation Plan

Car	From	To	Depart	Arrive	Minutes
2	2	4	01/01 00:00	01/01 04:00	240
3	3	5	01/01 00:00	01/01 03:30	210
4	5	3	01/01 00:00	01/01 03:30	210
4	4	3	01/04 19:30	01/04 22:30	180
5	4	2	01/01 00:00	01/01 04:00	240

Problem 2

For the large hidden instances, an exact IP model may be too slow. We therefore implemented a heuristic algorithm that always returns a feasible assignment and relocation plan within the three-minute grading limit. The submitted algorithm has two main stages. First, it quickly constructs two feasible plans using two different heuristics, called Algo1 and Algo5 in our implementation. It keeps the plan with the larger profit as the initial solution. Second, it applies a batch-based local integer programming improvement procedure. In each batch, a small number of cars are temporarily released, together with

the orders currently served by those cars, and a small IP is solved only for this local subproblem. The new local plan is accepted only if it improves the total profit and still satisfies the moving-time budget.

Algo1 is a greedy route-insertion method. Orders are sorted by high revenue first, then by earlier pickup time. For each order, the algorithm tries every compatible car, including the free upgrade rule that a level-l order may be served by a level-l or level-$(l+1)$ car. The order is inserted into the chronological route of the car if both adjacent transitions are feasible. Among all feasible insertions, Algo1 chooses the one with the smallest additional moving time, then the smallest upgrade penalty and idle time.

Algo5 is a demand-aware look-ahead heuristic. It divides the planning horizon into six-hour buckets and builds a future demand score for each station and car level. When assigning a car to an order, Algo5 scores the decision by combining the immediate profit benefit, relocation time, idle time, upgrade penalty, the future value of the return station, and the opportunity cost of removing the car from its current station. After this greedy pass, it performs several bounded repairs, such as shortage-bucket repair, route insertion, and one-removal replacement, to rescue valuable rejected orders without violating feasibility.

The final algorithm, implemented in `algo_union.py`, uses the following rule:

$$\texttt{pre\_build} = \arg\max\{\text{profit(Algo1)}, \text{profit(Algo5)}\}.$$

Because both initial heuristics are fast and feasible, taking their better plan is a low-risk way to combine two complementary decision rules. The resulting plan is then improved by `small_ip_improve`, which reuses the local IP repair idea from Algo4. A batch of cars is selected according to route efficiency. The orders on these cars are released; high-revenue compatible orders are added as candidates; then a local network-flow IP is solved over this small set. The IP uses the same type of start variables, link variables, and acceptance variables as in Problem 1, but only for the selected cars and candidate orders. The local repair is controlled by a wall-clock time limit, so the algorithm can safely stop and return the best feasible plan found so far.

Problem 3

The grading rule gives each program at most three minutes per instance. We therefore designed the algorithm as two parts: a very fast *pre-build* stage and a time-limited *optimization* stage. The pre-build stage must quickly return a feasible solution, while the optimization stage spends the remaining time improving only promising neighborhoods of that solution.

Pre-build. Our team proposed two construction approaches. We run both and keep the one with the larger profit.

- **Algo1: revenue-first insertion.** Orders are sorted by decreasing revenue and then by pickup time. For each order, Algo1 checks every compatible car, including the allowed one-level upgrade. It tries to insert the order into the car's chronological route and accepts the feasible insertion with the smallest extra moving time; ties are broken by smaller upgrade penalty and smaller idle time. If the average route length is $\bar{r} = n_K/n_C$, the insertion search costs about $O(n_K n_C \bar{r})$ in the worst case, and is much faster in sparse routes.
- **Algo5: demand-aware look-ahead.** Algo5 first builds six-hour demand buckets for each station and car level. The score of assigning a car to an order combines immediate profit gain, relocation time, idle time, upgrade penalty, future demand near the return station, and the opportunity cost of removing the car from its current station. It then performs bounded repairs such as shortage-bucket rescue, route insertion, and one-removal replacement for valuable rejected orders. Building demand scores costs $O(n_K + n_S n_L Q)$, where Q is the number of time buckets. The assignment scan is $O(n_K n_C)$, and the bounded repair loops have fixed caps in the implementation, so the running time remains well below the three-minute limit.

Batch IP optimization. Since pre-build is fast, we use the remaining time to solve many small exact subproblems. We convert the selected assignment into car routes. In each iteration, we sample a small batch of cars whose routes have relatively low efficiency, release their currently served orders, add a capped list of high-value compatible candidate orders, and solve a local network-flow IP for only those cars and orders. The local IP has the same meaning as the formulation in Problem 1: start variables assign a first order to a selected car, link variables connect two consecutive orders, and acceptance variables decide which candidate orders are served. The repaired batch is accepted only if it increases profit and keeps the total moving time within B. Otherwise, we keep the old feasible solution. This "release-and-repair" design lets us benefit from exact optimization without building the full IP.

The whole procedure is summarized below.

Input: one instance file, time limit T.
1. $(A_1, R_1) \leftarrow$ Algo1(instance).
2. $(A_5, R_5) \leftarrow$ Algo5(instance).
3. $A \leftarrow$ the assignment among A_1, A_5 with larger profit.
4. **while time remains do**
5. sample a small batch of cars from current routes;
6. release their served orders and build candidate order set;
7. solve the local IP with a short per-IP time limit;
8. if the local solution improves profit, update the routes;
9. **return** A and the corresponding relocation list.

Let n_C be the number of cars, n_K the number of orders, and n_S the number of stations. The local IP batch contains b selected cars and at most m candidate orders. It has $O(bm)$ start arcs and $O(bm^2)$ order-to-order arcs in the worst case, hence $O(bm^2)$ binary variables and constraints. In our implementation, both b and m are capped, and each local IP has a

short time limit. The total running time is therefore controlled by the three-minute wall-clock limit rather than by solving one enormous model.

Problem 4

Following the project instruction, we designed a numerical study with explicit factors and scenarios rather than relying only on the five public instances. The random instances are generated by `data/generate_code.py`. The planning horizon always starts at 2023/01/01 00:00. For each instance, the number of stations is fixed at $n_S = 100$, the number of levels is fixed at $n_L = 10$, and the number of planning days n_D is sampled uniformly from 7 to 100. Hourly rates are generated as a strictly increasing sequence over levels. Moving time is symmetric, with $T_{ii} = 0$ and $T_{ij} = 90$ minutes for $i \neq j$ in the generated study instances.

Scenario	Main factor	Ratio $n_K : n_C$	Levels	Stations	Time/Budget
S1	Baseline	1:1	random	random	random, $B = 10^6$
S2	Ultra-low load	1:10	random	random	random, $B = 10^6$
S3	Low load	1:3	random	random	random, $B = 10^6$
S4	High load	3:1	random	random	random, $B = 10^6$
S5	Level mismatch	1:1	8:2 skew	random	random, $B = 10^6$
S6	Forced relocation	1:1	random	odd/even	random, $B = 300n_D$
S7	One hub	1:1	random	hub 1G	random, $B = 300n_D$
S8	Two hubs	1:1	random	hub 2G	random, $B = 300n_D$
S9	Weekend effect	1:1	random	random	weekend, $B = 10^6$
S10	One peak	1:1	random	random	one peak, $B = 10^6$
S11	Three peaks	1:1	random	random	three peaks, $B = 10^6$
S12	No relocation budget	1:1	random	random	random, $B = 0$
S13	Tight budget	1:1	random	random	random, $B = 30n_D$
S14	Moderate budget	1:1	random	random	random, $B = 300n_D$

For the final detailed experiment, we selected five representative hard scenarios: S6, S8, S10, S13, and S14. These cover spatial imbalance, hub returns, temporal bursts, and relocation-budget sensitivity. For each selected scenario we generated 128 random instances, for 640 instances in total. To make the exact benchmark computationally possible, these selected scenarios use $n_C = 100$ and $n_K = 100$; this size is large enough to stress relocation and timing decisions, but still small enough for the IP solver to finish on our experiment machine. Since the IP model is exact for these generated instances, its objective value is the optimal profit and serves as an upper bound on any heuristic solution.

We also compare against a lower-bound feasible heuristic called `algo_naive`. It sorts orders by pickup time, assigns each order to the feasible car requiring the least relocation time, and rejects the order if no car can serve it. Since it always returns a feasible plan, its profit is a valid lower bound on the optimum, but it intentionally has no look-ahead or repair logic. A good heuristic should clearly outperform this baseline. The optimality gap is reported as

$$\text{gap} = \frac{\text{IP profit} - \text{algorithm profit}}{|\text{IP profit}|}.$$

Lower is better, and zero means that the heuristic matched the IP profit.

Scenario	Naive mean gap	Naive std.	Union mean gap	Union std.
S6	59.88%	222.65%	0.00%	0.00%
S8	97.53%	550.77%	0.68%	5.36%
S10	27.60%	12.85%	2.16%	2.01%
S13	1063.61%	3967.26%	29.07%	140.57%
S14	60.83%	233.16%	1.10%	4.66%

The histogram figures are generated under the scenario-study result directory. It contains one combined figure and one histogram for each selected scenario. The full raw experiment, including the 640 generated instances, optimal plans, run-time logs, per-instance benchmark rows, and histograms, is stored in the compressed artifact `scenario_study_128_results.tgz`.

The results show that `algo_union` is consistently much closer to the IP benchmark than the naive chronological rule. On S6, the union heuristic matched the IP solution on all 128 instances. On S8, S10, and S14, the average gap remained below 2.2%, while the naive baseline had gaps between 27.6% and 97.5%. S13 is the hardest case because the relocation budget is very tight and the IP profit is often negative or close to zero, which makes percentage gaps large and volatile. Even in this case, `algo_union` reduced the average gap from 1063.61% for the naive baseline to 29.07%. This supports the main design idea: a fast greedy construction is useful, but local batch IP repair is crucial when relocation budget or spatial imbalance makes early greedy choices expensive.

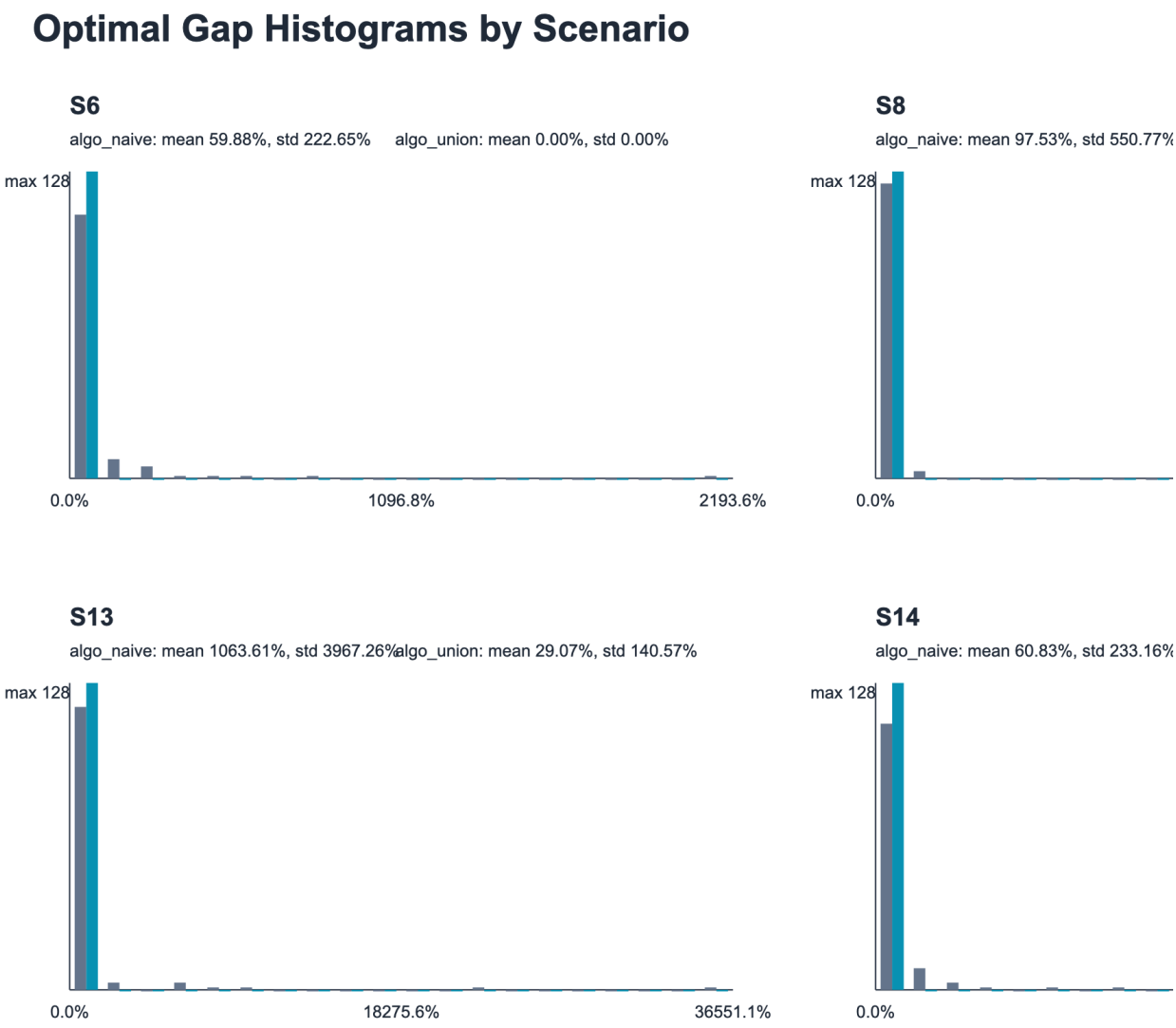

Figure 1: Optimality-gap histograms for the five selected scenarios. Each panel reports the mean and standard deviation of `algo_naive` and `algo_union`.